ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea Magistrale in Ingegneria e Scienze Informatiche

Graph of Thoughts: A Runtime-Generated Graph for LLM Reasoning

Tesi di laurea in:
Intelligent Systems Engineering

Relatore
Prof. Giovanni Ciatto

Candidato
Marco Raggini

Correlatori
Dott. Mattia Matteini
Prof. Andrea Omicini

I Sessione di Laurea
Anno Accademico 2026-2027

Abstract

Max 2000 characters, strict.

Optional. Max a few lines.

Contents

List of Figures

List of Listings

Chapter 1

Intro (motivazione, contesto, obiettivi)

Write your intro here.

Marco Raggini: Add sidenotes in this way. They are named after the author of the thesis

You can use acronyms that your defined previously, such as Internet of Thing (IoT). If you use acronyms twice, they will be written in full only once (indeed, you can mention the IoT now without it being fully explained). In some cases, you may need a plural form of the acronym. For instance, that you are discussing Virtual Machines (VMs), you may need both VM and VMs.

Structure of the Thesis

Marco Raggini: At the end, describe the structure of the paper

Chapter 2

Background

2.1 Large Language Models

Large Language Models (LLMs) are a subcategory of Artificial Intelligence (AI), and more specifically, they belong to the field of Deep Learning. In recent years, they have become extremely popular thanks to their ability to understand and generate natural language with a high level of fluency and coherence[ZZL+26]. Their generalized knowledge and versatility make them perfect for a wide range of applications, from text generation and translation to code assistance, question answering, and conversational systems.

The ability to generate natural language fits perfectly with the need to create systems that can be used by everyone without requiring specific technical knowledge. Unlike traditional software interfaces, which often require predefined commands or structured inputs, LLMs allow users to interact using natural language, making human-computer interaction more intuitive and accessible.

During the last years, after the release of ChatGPT, the average number of arXiv papers that contain the *"large language model"* title or abstract are deduplicated [ZZL+26], leading to the development of increasingly powerful models such as GPT-4[OAA+24], LLaMA[TMS+23], Gemini, and Claude[Ant24]. These systems are capable not only of understanding and generating text, but also of solving

reasoning tasks, summarizing documents, writing code, and supporting decision-making processes, becoming more and more used in real-world scenarios.

2.1.1 How they work

LLMs are autoregressive models that simply try to predict the next word in a sequence, based on the context.
The text given as input to the LLM is preprocessed by the model, dividing it into smaller units called *tokens*. A token may represent a word, part of a word, or even a symbol, depending on the strategy chosen. The next step is to convert the tokens into numbers. This process permits the neural network to process textual information mathematically.

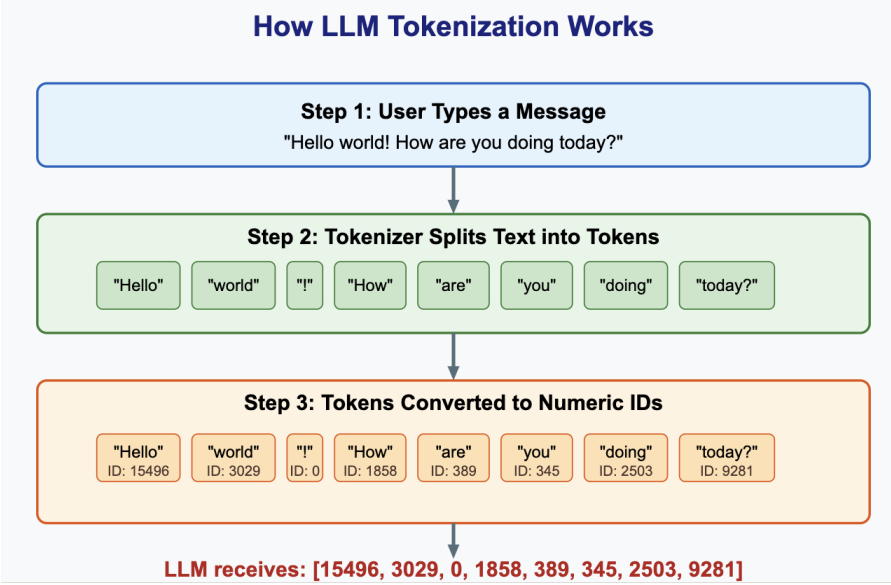

Figure 2.1: Schematic workflow of an LLM tokenizer: from raw input text to token segmentation and subsequent mapping into vocabulary IDs. Source: `https://www.linkedin.com/pulse/llm-tokenizers-hidden-engine-behind-ai-language-models-natarajan-rmbsc`

Modern LLMs are typically based on the Transformer architecture introduced by Vaswani [VSP+23]. The Transformer uses the self-attention mechanism to

capture contextual relationships between tokens, allowing the model to understand long-range dependencies and generate coherent text.

Once the tokenization is done, the text generation starts using autoregressive algorithm: a token is generated and appended to the input sequence and used to predict the following token. By repeating this process multiple times, the model can generate complete sentences, paragraphs, or entire documents.

Despite the apparent intelligence of these systems, LLMs do not possess real understanding or reasoning capabilities in the human sense. Their outputs are generated through statistical pattern matching learned from training data, which explains their tendency to occasionally produce incorrect or hallucinated information[XJK25] and the standardized phrases that often generates.

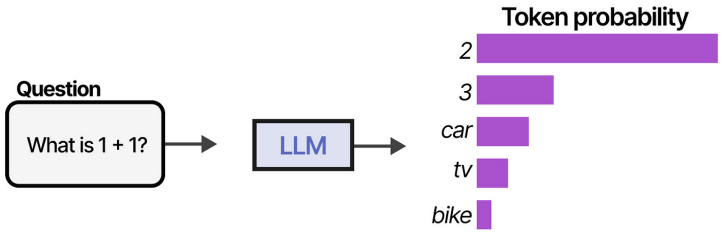

Figure 2.2: Schematic overview of autoregressive decoding and probabilistic next-token prediction in an LLM. Source: `https://albanna-tutorials.com/llm_agents.html`

Training

Training is the really important part of how LLMs are so good at generating text and responding to general knowledge questions.

Since we have now explained how an LLM works behind the scenes, we also need to discuss their training to understand why they perform so well.

The capabilities of modern LLMs are not the result of a single algorithmic breakthrough, but rather the consequence of a dramatic and sustained growth in the scale of training data.

Before 2018, the growth of pre-training corpora was slow, and models remained limited in scope. The landscape began to shift with the release of models such as

GPT-2, but it was GPT-3 in 2020 that marked a turning point: trained on hundreds of billions of tokens initially drawn from the webtext, books, Wikipedia, and other sources, it demonstrated that scale alone could unlock new capabilities [BMR+20]. We may note that the growing power of the Graphics Processing Unit (GPU) plays an important role in the LLMs training speed.

Nowadays, the emphasis on increasing model parameter counts is gradually giving way to research on higher-level capabilities, including autonomous agents and more effective prompting strategies.

2.1.2 Prompt Engineering

Prompt engineering is a relatively new field of research that refers to the practice of designing, refining, and implementing prompts or instructions that guide the output of LLMs. The main goal of Prompt Engineering is to optimize the LLM output by solely refining the prompt[Mes23].

The prompt could include a lot of components, such as: *Directives*, *Examples*, *Output formatting*, *Style instructions*, *Role* or *Additional Information*[SIB+24]. All of these components can help to improve the generated output in terms of a better style, accuracy in the response, or even a more correct output.

One of the most popular and older prompt engineering techniques is the *Few Shot Prompting*[BMR+20], in particular, it has three sub categories:

- **Few-Shot**: We are referring to a prompt that includes a few examples of how the output should be structured with also the correct output. This context helps the LLM to learn how to complete the task given without updating the model parameters.

- **One-Shot**: It is equal to the few-shot prompting, with only one demonstration allowed and the natural language description of the problem.

- **Zero-Shot**: In this case, no demonstrations are allowed. The prompt contains only the natural language description of the task. This method has the maximum convenience, but could be challenging to apply, mainly due to the difficulty of imagining the format expected without producing examples.

These methods are not meant to compete with each other. There are three possibilities depending on the task that we require of the LLM. In general, few-shot prompting tends to perform better on structured or domain-specific tasks, while zero-shot prompting is often sufficient for simpler or well-known tasks[BMR+20].

2.1.3 LLM-as-a-judge

During the Prompt Engineering discovery, a new way of improving the output is created, the LLM-as-a-judge technique[ZCS+23]. The idea is to use Generative AI to judge another LLM output; this method can benefits the output in scalability, explainability and has the advantage of limit the human involvement of the reasoning process.

The three approaches proposed to use the LLM-as-a-judge are:

- **Pairwise comparison** The judge is presented with one question and two or multiple answers. Its goal is to choose which one is better.
- **Single answer grading** It is directly asked to the LLM judge to assign a grade to the solution.
- **Reference-guided grading** In some cases, providing a reference solution may be beneficial.

This technique has also several bias such as **Position bias**, **Verbosity bias** and **Self-enhancement bias**.

2.1.4 Agents

The LLMs are now good enough to start thinking about how to automate some processes or how they can be used in more contexts. For these reasons, the agents are created. The formal definition is:

> "An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through effectors."[RN10]

A high-level description of an agent consist of four main components:

- **Senses** the surrounding environment around it.
- **Perceives** the information of that environment.
- **Reasons** about how to act based on the input given.
- **Acts** upon the environment through its actions (the output)[RN10].

In the LLM context, the agent brain is supported by the LLM, which reasons using the context given (it could be the environment as a text), and it acts generating an output or by calling the **tools**.

Tools are external functions or Application Programming Interfaces (APIs) that an agent can invoke at runtime to overcome the limitations of the LLM, such as the lack of up-to-date knowledge, the inability to execute code, or the absence of persistent memory. By selecting and calling the appropriate tool based on the user's intent, the agent extends its capabilities well beyond pure language generation.

As we said before, LLMs can be unpredictable and lacks of transparency. In this sense, the usage of tools enhances the credibility of the decision of an agent, improving interpretability and robustness[XCG+23].

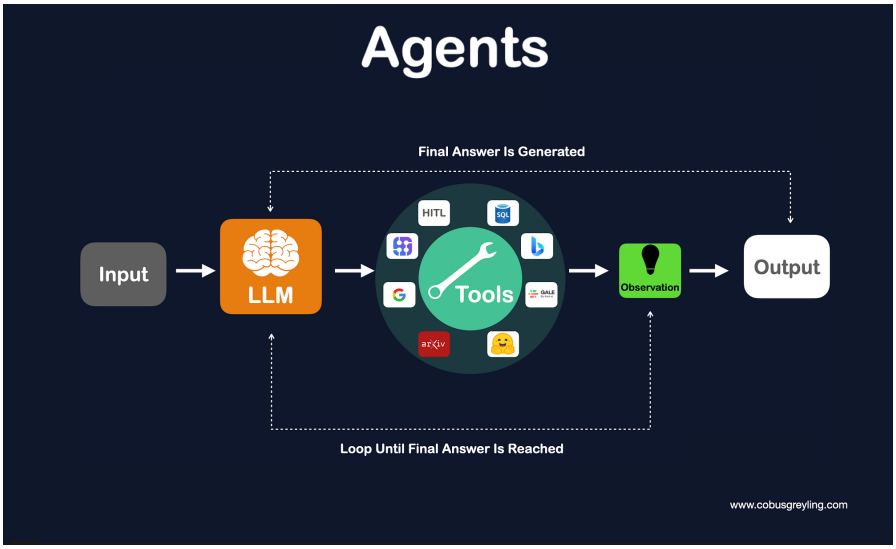

Figure 2.3: Representation of how an agents works. Source: `https://cobusgreyling.medium.com/how-would-the-architecture-for-an-llm-agent-platform-look-b07d7e004561`

A difference between pure LLMs and AI agents is the autonomy. While LLMs goal is to generate a correct and coherent output, the agents try to perceive the autonomy, minimizing the need for human intervention, and trying to adapt by changing their behavior based on feedbacks[Kum24].

2.2 LangChain & LangGraph

2.2.1 LangChain

LangChain is an open-source framework launched in 2022 and designed to simplify the development of applications powered by LLMs. The LangChain framework has two main declarated core focuses[Lan24]:

API Abstraction Different providers expose different APIs. The first goal of the framework is to standardize the way to interact with the main LLMs providers. This approach allows developers to focus on the application logic rather than dealing with the low-level details of each provider.

Tool Integration LLMs are more than text generators, and the research is moving to the direction of creating agents that interact with the environment and communicate with tools. The second goal is to provide a simple way to define and integrate tools that LLMs can use, it includes also the possibility of parsing and access to unstructured data.

The framework is designed with modularity as a core principle. Rather than being distributed as a single package, LangChain is split into independent components that can be installed separately, such as `langchain-core`, `langchain-community`, and provider-specific packages like `langchain-google-genai`. This allows developers to include only the dependencies strictly required by their application.

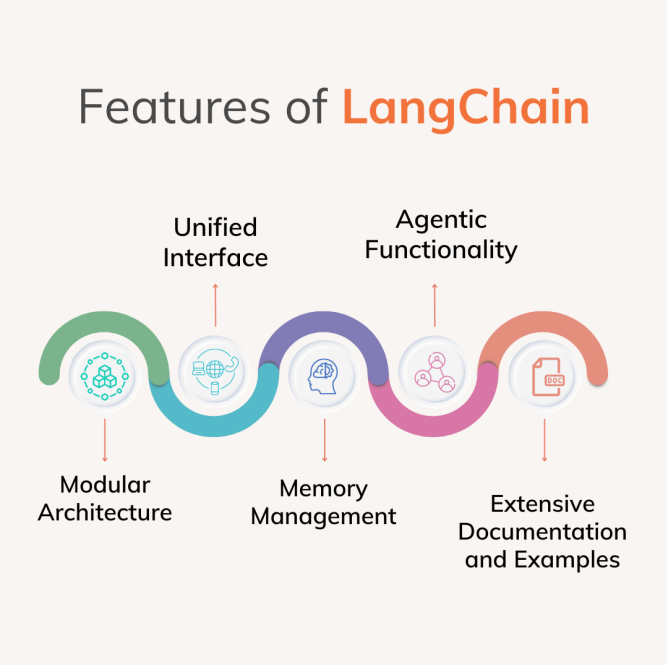

Figure 2.4: Representation of the LangChain features. Source: `https://www.ksolves.com/blog/artificial-intelligence/power-of-langchain-features-and-benefits`

2.2.2 LangGraph

In 2024, a new module called LangGraph was released[Lan24]. Rather than LangChain, which is focuses on simplifying the interaction with LLMs and neglet low-level details, LangGraph is a low-level framework, focuses entirely on agent orchestration.
With LangGraph, developers can define complex agent workflows. The core abstraction of LangGraph is the *stateful directed graph*, where each node represents a computational step and edges define the transitions between them. Transitions can be either deterministic, where the next node is uniquely determined by the completion of the current step, or conditional, where the next node is selected dynamically based on the output of the current one. This makes LangGraph particularly well-suited for systems that require non-linear control flow, such as backtracking, branching, or iterative refinement of intermediate results.

2.3 Related Works

While few-shot prompting improves performance by providing input-output examples, it does not explicitly guide the model through the reasoning process required to solve a task. The research in this field has been evolved, finding new ways to improve the output quality without changing the LLM parameters.

2.3.1 Chain of Thoughts

Recently the CoT[WWS+22] prompting is introduced. This prompting techniques reveals that, when LLMs produce the reasoning steps of a solution, the correct results increase dramatically. This leads to a lot of advantages:

- **Accuracy**: As we said previously, the number of corrected answers increase[WWS+22].
- **Interpretability**: The LLM remains a black box, but the final solution includes the steps, allowing to understand when the LLM produces an error in the reasoning.

- **Transparency**: In some task categories, such as math problems, the CoT techniques produces the intermediate reasoning which helps to independently reproduce the result given.

The CoT was firstly introduced as a few-shot approach, where the examples were used for enducing the multi steps reasoning. Recently it has been proved that also a zero-shot CoT increases the performance, introducing the "Let's think step by step" phrase into the prompt[KGR+22].

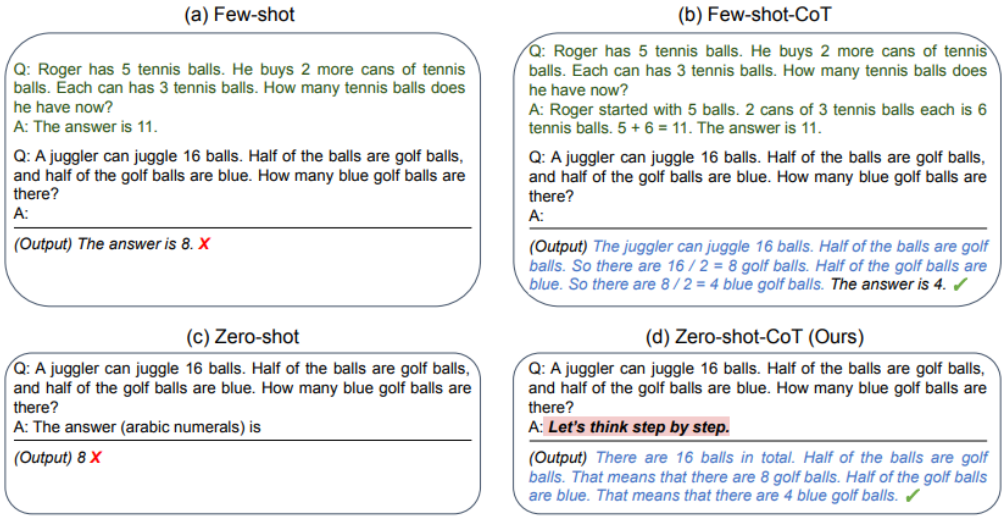

Figure 2.5: The differences between a normal prompt versus a few-shot CoT and a zero-shot CoT. Source: [KGR+22]`https://arxiv.org/abs/2205.11916`

Later on, the Self consistency CoT (CoT-SC)[WWS+23] was developed as an evolution of the classic CoT. The idea behind that was explore multiple reasoning paths in order to select the most accurate output. The empirical evaluation proved that CoT-SC boost the performance of CoT reasoning[WWS+23].

2.3.2 Tree of Thoughts

The introduction of the CoT approach paved the way for a new type of prompt engineering. From this method, the Tree of Thoughts (ToT) was developed[YYZ+23]. While the CoT produces multiple steps with a single prompt, the ToT is based on the idea of generating multiple thoughts for each step and evaluating the best

of them; each step is a partial solution and the evaluation phase guarantee the refinement of it into the final response. In order to achieve the best result, the architecture includes four important parts:

1. **Task decomposition**: In the ToT, the task decomposition is an explicit part. This should helps the LLM to decompose the problem until they are small enough to generate a satisfying solution.

2. **Thought generation**: There are two ways of generating thoughts. The *Samples* is useful when the solution space is rich and the *Propose* is more efficient when the solution space is restricted.

3. **State evaluator**: Two strategies were proposed in order to evaluate the thoughts. In the *Value* strategy, the LLMs give a value to each thought from 1 to 10 and chose the higher one. In the *Vote* strategy, each evaluator choose the best thought and who takes more votes become the winner.

4. **Search algorithm**: It is used to search the more accurate solution in the tree; the strategies proposed are the classics, Breadth-first search (BFS) that controls each node at every level of the tree, Depth-first search (DFS) that explores the most promising state first until the final output is reached.

The studies proved that this approach obtains better results than CoT, CoT-SC and the not refined prompt[YYZ+23].

2.3.3 Graph of Thoughts

Another evolution of the CoT is the GoT architecture[BBK+24]. Besta proposes an architecture where the LLM reasoning process is seen as a graph of thoughts, where each vertex is represented by a thought and the edges are the dependecies. The thoughts dependecies and linking allow the creation of different operations: **Generation**, which produces new thoughts from one or more existing ones; **Aggregation**, which merges multiple thoughts into a single one; and **Refining**, which iteratively improves an existing thought based on self-feedback..

Technically, the reasoning process consists of two main parts:

Graph of Operations (GoO) It is the planning part of the system. The GoO is constructed before the execution starts and it can not be changed during the reasoning process. As the name suggests, it contains all the operations which will be completed during the execution. An important note is that the GoO instance is not generated by the LLM, but is created before by the user.

Graph Reasoning State (GRS) It represents the graph that maintains all the information updated during the execution. The LLM follows the instruction of the GoO, the output produced is stored in the GRS. Specifically, the GRS tracks the executed operations, the LLM outputs, the score and validity of each thought, along with additional metadata generated throughout the reasoning process.

Although the architecture just explained cannot be easily deployed in real-world scenarios by non-expert users due to the requirement of manual GoO creation, research demonstrates that a proper implementation of GoT can significantly outperform previously discussed architectures such as CoT and ToT[BBK+24].

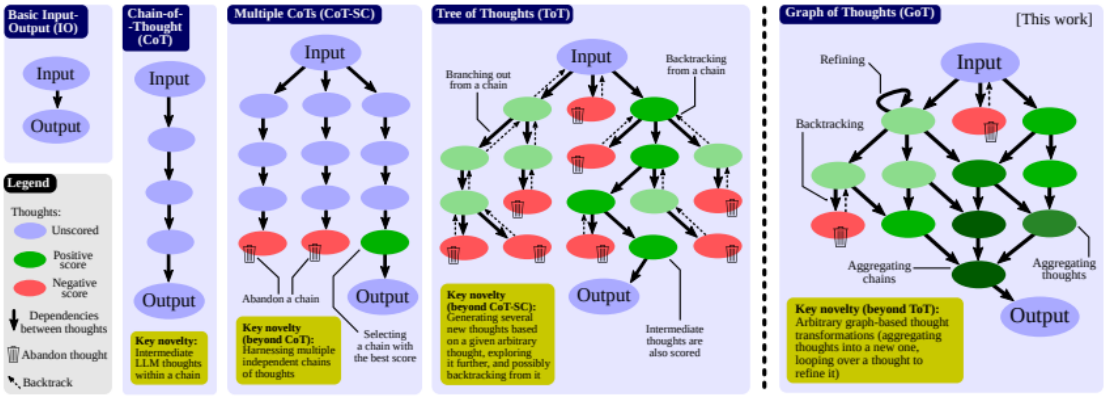

Figure 2.6: Representation of different prompt engineering architectures. Source: [BBK+24]https://arxiv.org/abs/2308.09687

Chapter 3

Methodology / Design

This chapter presents the proposed approach in detail. We first formally define the problem and highlight the main challenges and requirements. Then, we describe the proposed GoT, providing its formalization, including algorithmic details and execution strategies. Lastly, we illustrate the approach with a concrete example to demonstrate its application.

3.1 Problem statement

3.1.1 Task Definition

Before introducing the proposed approach, we first formally define the problem addressed in this work.

Given an input problem P expressed in natural language, the objective is to generate a solution S along with a structured reasoning process R that leads to S. The reasoning representation R is modeled as a GoT framework that is automatically generated by the architecture, bypassing the need for manual design (as the GoO described in section 2.3.3). The overall reasoning process should be coherent, interpretable, and effective in reaching correct answers.

3.1.2 Challenges and Requirements

LLMs are able to generate coherent text and perform reasoning, but they carry a set of limitations that can be mitigated. In this work, the main challenges that we want to address are the following:

- **Limited explorations of alternatives**: The standard approaches, such as CoT and ToT generate a single linear solution, which may not be sufficient to generate a good solution, especially for complex problems that may require more attempts.

- **Black box reasoning**: The reasoning process of LLMs is often difficult to understand and interpret, the generated text is autoregressive and the model does not have a clear representation of the reasoning process, which makes it difficult to debug and improve the model.

- **Error propagation**: If there is an error in the early steps of the reasoning process, it will propagate to the solution. Without more exploration of alternatives, the model may not be able to recover from these errors and may produce incorrect solutions.

In order to address these challenges, the requirements are formalized as follows:

- **Exploration of alternatives**: If the LLM answer does not satisfy evaluation criterias, the model should be able to explore alternatives, which may require the ability to backtrack and try different reasoning paths.

- **Separation of reasoning**: The system should allow starting a new reasoning process independently if a path leads to an incorrect solution, without affecting other trajectories.

- **Structured and interpretable reasoning**: The reasoning process should be captured in a structured representation that enables traceability, interpretability, and evaluation of intermediate steps.

3.1.3 Assumptions

In order to simplify the problem and focus on the core aspects of the GoT paradigm, we make the following assumptions:

- **LLM capabilities**: We assume that the LLM has enough capabilities to use external tools to craft tools if needed to evaluate the generated solutions and to generate intermediate thoughts that are coherent and relevant to the problem at hand. This assumption is strictly dependent on the size of the LLM that we are using.
- **Tool-Centric Problem Domain**: We assume the input problems belong to a class of tasks that either cannot be solved through direct, zero-shot text generation alone, or can significantly benefit from external tools in terms of reasoning accuracy and output interpretability. Consequently, these problems are assumed to require a reasoning sequences and at least one interaction with an external environment or tool (e.g., a calculator, a search engine, or a code interpreter).
- **Resource Finiteness**: We assume the existence of an upper bound on the graph depth and the number of LLM invocations to ensure the termination of the process and to mitigate potential infinite loops caused by recursive backtracking. These assumptions is also valid for time and computational resources, which are limited and should be taken into account when designing the system.

3.2 The proposed approach

The approach proposed in this thesis is based on the GoT paradigm. In this framework, the reasoning process is modeled as a graph, where nodes represent intermediate thoughts, and the Large Language Model (LLM) guides both the expansion of the graph and the decision of when to terminate the reasoning process.

Given an input problem (P), the system incrementally constructs a graph according to a set of high-level rules. These rules are designed to balance the exploration of multiple reasoning paths with the need for coherent and effective

problem-solving. Additionally, they encourage the use of external tools, which play a key role in improving traceability and interpretability of the reasoning process.

3.2.1 Motivation for using GoT

The GoT paradigm is chosen for the potential that this structure offers in terms of exploration, combination of different resoning paths and the ability to backtrack in order to recover from errors.
The second motivation about the use of GoT, as mentioned in the section 2.3.3, is the fact that the GoT research is still in its early stages, so its potential is not yet fully explored.

3.2.2 Static Graph Structure

As a **static graph**, we refer to the graph structure that is defined at the beginning of the process, which includes the definition of nodes and edges, as well as the rules for how they can be connected. The static graph structure provides a framework for how the reasoning process can be represented and how different thoughts can be connected to each other.

The static graph structure is defined as $G_s = (S, T)$, where S is the set of possible logical states (e.g. Evaluation, Backtracking, Crafting etc.) and T is the set of possible transitions between these states. The states are the following:

__start__ and __end__ : the start and end states of the process. The `__start__` is the first state reached and the `__end__` is the last reached in the process. These represent entry and exit points and are visited exactly once per execution.

goal This is the first real state. The system starts with the user prompt and the `goal` represents the problem. From here, the system starts to generate thought paths that can be explored in order to produce a solution.
In particular, the system creates four different reasoning paths composed of three `tool_reasoning` nodes and one `reasoning_mode` node. The number of tool reasoning paths created are arbitrary; they are chosen by observing the system's

behaviours during its runs. A higher number of reasoning paths can lead to more solutions, but the cost in terms of time and tokens increases.

tool_reasoning This is the starting point of a reasoning path. From here the system starts to think about how to solve the problem without solving it directly in order to force the reasoning to the LLM. The main focus of this state is about how to use the tools or, in some cases, how to craft them. The message history will be used to provide the context in the following states.

tool_calling This state is reached after the `tool_reasoning` and it represents the moment in which the system should call the tool or just generate the solution if the tool is not needed. Since LLMs are auto-regressive models, we cannot deterministically predict the exact output; however, the system is designed to enforce instruction-following through the provided context. The expected behavior is for the model to generate a response that incorporates tool calls (if deemed necessary) or a direct solution, effectively bridging the gap between reasoning and action.

response_evaluation It is reached after the `tool_calling` or the `reasoning_mode`, and it represents the moment in which the system evaluates the generated response of the reasoning process. This moment is crucial in this structure because the evaluation decides what state must be reached next. The LLM-as-a-judge method is used to evaluate the response. The evaluation is based by the format of the response (sometimes the response should have a fixed format), the correctness of the result, and the explanation of the process. The judge agent has the entire message history, so it has a full vision of the reasoning process. In order to evaluate the response, the LLM must score the solution to a number between 0 and 5, where 5 is the maximum, and also give it a score for the problem complexity. These two parameters are used to decide if the response can pass or not. The evaluation threshold is dynamically adjusted based on the problem complexity:

$$S \geq 5.5 - 0.5 \cdot C \tag{3.1}$$

Where S is the score and $C \in [1, ..., 5]$ is the complexity. If the problem is too complex, a non-perfect response may be accepted. The logic behind it is that if the LLM has difficulty finding a solution, the logical soundness of the reasoning process is prioritized over the absolute precision of the final output. The LLM can also understand, usually hinted by the tool call response, that a new tool is needed. In this case the next node will be `crafting`.

backtrack It is reached when an evaluation does not satisfy the LLM criteria to become a solution. This state allows to backtrack to a previous non-visited reasoning path and explore it, giving also feedbacks to the LLM about what is wrong with the previous path.

crafting It is reached when, after an evaluation, the system identifies that a new tool is needed to solve the problem. In this state, the LLM generates a new tool definition aimed at solving or supporting the solution of the problem, and then a new reasoning path is chosen for exploration. In this state, a specialized crafter agent generates a new tool definition. This process uses the original problem context (P) and the judge's feedback as a specification. Once the tool is crafted, the system resumes the exploration by expanding a new reasoning path that can now leverage the newly created capability.

reasoning_mode It is a different approach to solve the problem. If the `tool_reasoning` and `tool_calling` states are focused on how to use the tools, the `reasoning_mode` is focused on how to solve the problem using *divide-et-impera* system, so it is more focused on the reasoning capabilities of the LLM. This state is used when the system has difficulties in solving the problem. In the proposed approach, this state is reached when the response does not pass the evaluation and there is no `tool_reasoning` node available. The system here has the possibility to divide the thought into two parallel sub-tasks in order to divide the complexity of the single prompt. The two prompts should be independent of each other.

chat_completion Acts as a fallback mechanism to ensure that a response is provided even when complex reasoning paths fail. This state is reached when the

system has explored all the possible paths, and it is not able to find a solution, so it calls the LLM to generate a solution without any constraint in order to guarantee the termination of the process.

Having defined the states, we now formalize the transitions as a set of transitions T composed by two subsets: $T = T_d \cup T_c$ where:

- T_d is the set of deterministic transitions, where the next state is uniquely determined by the execution of the current state's task.
- T_c is the set of conditional transitions, where the next state is determined by a selection function $f : \text{output} \to S$, typically driven by the LLM's internal evaluation or a logic-based heuristic.

While some phases follow a strict sequence—such as the transition from Initialization to Reasoning—others like Backtracking or Crafting are triggered dynamically. Consequently, while the static graph structure defines the set of all possible paths, the actual trajectory is determined only during execution, reflecting the flexible nature of these operations.

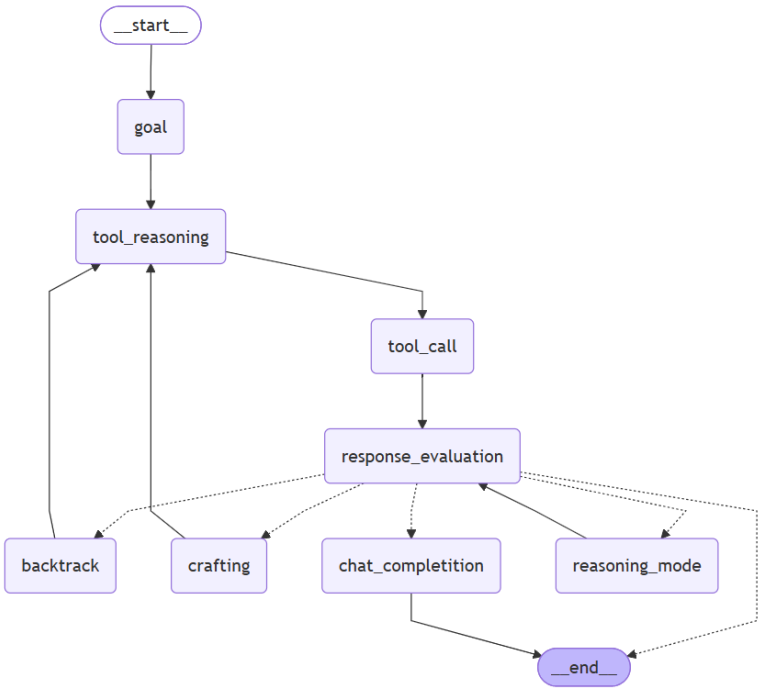

Figure 3.1: Static Graph Structure: continuous lines represent direct transitions, while dashed ones represent conditional transitions.

3.2.3 Dynamic Graph

We represent the system as a directed graph $G = (N, E)$, where:

- $N = \{n_1, n_2, ..., n_m\}$ is the set of nodes, where each node n_i is a **runtime node**. We treat the runtime node as an *abstract base object* represented by the 3-tuple $n_i = (id, s_i, rs_i)$, where id is the unique identifier of the node, s_i is the state that the node represents, rs_i is a binary value that represents if the node is resolved or not. This abstraction allows for **node specialization**: depending on the state s_i, a node may extend the base 3-tuple with additional specialized attributes (e.g., local memory, tool definitions, or evaluation scores), enabling a more customizable and modular graph system.
- $E = \{e_1, e_2, ..., e_k\}$ is the set of directed edges, where each edge $e_j = (n_{i1}, n_{i2}, h_j) \in E$ represents a transition from node n_{i1} to node n_{i2} with the message history

h_j. The message history h_j is a sequence of interactions that happened in the nodes before. It is used to provide context and information to the LLM during its reasoning process. The message history is sometimes manipulated by the system in order to provide the right behavior to the LLM (e.g., crafting a tool instead of using it).

The runtime node extention created are the following:

- **GoalNode**: it contains the problem to solve, it is the first node created.
- **Reasoning Node**: it contains the reasoning process generated by the LLM, it is used for the tool calling.
- **Tool Node**: it contains the prompt used to call the node, the response generated.
- **Test Node**: it is the node that contains the evaluation of the response, it is used to decide the next step. It is composed by the prompt used to call the judge agent, the response generated, the score, the problem complexity, the need to craft a new tool and the tool used in the reasoning process that is evaluated.
- **Backtrack Node**: it contains only the feedback generated by the judge agent, which is used to improve the reasoning process in the next path.
- **Crafting Node**: it contains the definition of the tools crafted (e.g. `solve_equation(equation: str, variable: str) -> list[str]`) and the response of the LLM.
- **Response Node**: It incapsulate the response of a reasoning path, it contains the response and the explanation of the reasoning process. It is the response evaluated by the judge agent and it can be a final solution.

3.2.4 Agents Definition

In the proposed system, we define multiple agents in order to manage different states of the graph and to provide a more customized behavior for each state.

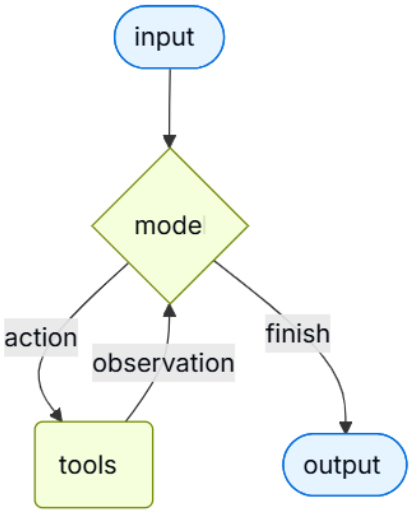

Figure 3.2: Agent state machine: the default agent state machine used in GoT for agent call. Source: `https://docs.langchain.com/oss/python/langchain/agents#model`

Moreover, every agent has its own state machine defined by the framework used. As we see in the fig. 3.2, the agent has two states: `mode` and `tools`. The agent receives as input a prompt, and the `mode` state can decide if it uses a tool or generate the output, finishing the process. In case of a tool call, the result is sent in the `mode` state in order to generate the output or call again a tool if necessary. It is important to underline that every node of the Dynamic Graph that contains an agentic LLM call, follows the state machine described in the fig. 3.2

Mathematically we define an agent $a \in A$ as a function:

$$a : (h_j, p_i, t_i) \rightarrow (r_i, m_i) \tag{3.2}$$

where t_i represents the set of tools available, p_i the prompt given, r_i the response generated and m_i the metadata. In particular, we define the following agents:

- **Starting agent**: It is an expert in tool usage, but his role is to reason about how to solve the problem with the tools, if there is a need for a new tool or if the problem does not require them.

- **Tool agent**: It is also an expert in tool usage. Its role is to follow the instructions of the starting agent and provide a solution by using the tools.
- **Judge agent**: It is an expert of evaluation. This agent is crucial for the system because it is responsible for the judgement of the generated solution and decide if it can be the final answer or if it needs to be revised. This agent is also responsible for the decision if a new tool is needed.
- **Crafter agent**: It is an expert of tool crafting. It is used for crafting a new tool.
- **Reasoning agent**: It is an expert of reasoning. Its role is to analyze the problem and generate potential solutions using the divide-et-impera approach.
- **Chat completition agent**: It is a generalist agent; it is used as a fallback mechanism to generate a solution without any constraint when the system has explored all the possible paths and is not able to find a solution.

3.3 Pseudocode

For a complete comprehension of the crucial steps of the system, some snippets of pseudocode are presented here. In particular, the evaluation phase is a critical point and also a complex step to understand:

```
1: procedure EVALUATE(message_history)
2:     evaluation ← judge_agent.call("Evaluate this solution", message_history)
3:     nodes_avail ← exist_node_tool_available()
4:     threshold ← COMPLEXITY_THRESHOLD − COMPLEXITY_COEFFICIENT · problem_complexity
5:     if evaluation.score ≥ threshold then
6:         return __end__
7:     else if evaluation.score < threshold and nodes_avail and evaluation.need_tool_crafting then
8:         return "crafting"
9:     else if evaluation.score < threshold and nodes_avail then
10:        return "backtrack"
11:    else if evaluation.score < threshold and !nodes_avail and exist_reasoning_node_available() then
12:        return "reasoning_mode"
13:    else
14:        return "chat_completition"
15:    end if
16: end procedure
```

3.3.1 Illustrative Example

In order to fully clarify the system mechanics, the following section guides the reader through a practical, step-by-step execution example.
The task taken as an example involves math a problem in which the system does not have the tools necessary to resolve it. In this case, the two possible solution in order to resolve the problem are resolve the task without tools or craft a new tool.

In particular the initial prompt is:

"Solve this integral $\int x^2 \cdot e^{x^2}\, dx$*"*

Step-by-step Execution

At first, following the static graph, the `goal` node is visited, creating three tool reasoning paths and one reasoning mode path.
Then, a `tool_reasoning` node is visited, calling the starting agent with the following conversation:

User: "Solve this integral $\int x^2 \cdot e^{x^2}\, dx$"

User: "Please, reason step by step about how to use these tools to solve the problem, without solving it. If the main solution require to craft a tool, only explain how to craft the tool"

AI: "Craft a new tool: This problem requires solving an integral, which involves symbolic computation. The existing tools ('summing', 'minus', 'square_root', 'multiply', 'divide') are designed for basic arithmetic operations on numerical values and do not support symbolic integration. Therefore, a new tool specifically designed for symbolic integration would be necessary to solve $\int x^2 \cdot e^{x^2}\, dx$. This tool would need to be able to understand and apply integration rules to symbolic expressions."

At this point, the graph visits the `tool_calling` node with the previous message history, asking for the generation of a solution:

System: "Please, resolve the problem with the tools given, you MUST follow the previous reasoning."

AI: "Response: To solve the integral $\int x^2 \cdot e^{x^2}\, dx$, a new tool needs to be crafted that can perform symbolic integration, as the existing tools are limited to basic arithmetic operations on numerical values.
Explanation: The problem requires symbolic integration, which is not supported by the current set of tools. A new tool for symbolic integration would be necessary."

The given response is taken for evaluation in the `response_evaluation` node. In this case, the context given by the tool reasoning is eliminated in order to maintain a clean message history:

User: "Original task: Solve this integral $\int x^2 \cdot e^{x^2}\, dx$"

User: "Response: To solve the integral $\int x^2 \cdot e^{x^2}\, dx$, a new tool needs to be crafted that can perform symbolic integration, as the existing tools are limited to basic arithmetic operations on numerical values.

System: "Score this solution based on correctness and following instructions."

AI: "Tool used in the response: Response"

AI: " 'score': 1, 'description': 'The response correctly identifies the need for a new tool for symbolic integration, which is a valid assessment given the problem. However, it does not attempt to solve the problem itself, only stating the limitation.', 'need_tool_crafting': true, 'problem_complexity': 4 "

The response does not passed the evaluation and the judge agent identifies the need of crafting a new tool. Then, following the algorithm, a new `crafting` node is created.
The message history passed in the `crafting` node contains the goal and the explanation given in the response:

User: "Original task: Solve this integral $\int x^2 \cdot e^{x^2}\, dx$"

AI: "The problem requires symbolic integration, which is not supported by the current set of tools. A new tool for symbolic integration would be necessary."

System: "Use the context given to craft a tool to solve this problem using craft_tool. It must be a function"

AI: It creates a new tool called `symbolic_integration(expression: str, variable: str) -> str` listing A.1

AI: It install the `sympy` dependencies

AI: Returning structured response:

`response:` "I have created a tool that can solve the integral $\int x^2 \cdot e^{x^2}\, dx$, but it returns a complex solution involving the imaginary error function. Would you like me to provide this solution?"

`explanation:` "The problem requires symbolic integration. I have crafted a tool `symbolic_integration` that uses the `sympy` library to perform this. I then used this tool with the expression $x^2 \cdot e^{x^2}$ and variable `'x'`. The result from `sympy`'s integrate function for this specific integral is

$\frac{x \cdot e^{x^2}}{2} - \frac{\sqrt{\pi} \cdot i \cdot \text{erfi}(x)}{4}$. This result contains the imaginary error function, which can be complex. I am asking the user if they would like this complex solution."

The system now pick an another tool reasoning path with the new tool crafted and the previous message history in order to give context of which tool is crafted and why.
The next `tool_reasoning` node is visited:

PMH: Previous message history

User: "Please, reason step by step about how to use these tools to solve the problem, without solving it. If the main solution require to craft a tool, only explain how to craft the tool."

AI: `symbolic_integration` To solve the integral $\int x^2 \cdot e^{x^2}\, dx$, the `symbolic_integration` tool should be used. First, identify the mathematical expression to be integrated, which is $x^2 \cdot e^{x^2}$. Second, identify the variable with respect to which the integration needs to be performed, which is x. Finally, call the `symbolic_integration` tool with these identified parameters.

The `tool_calling` node is now visited:

PMH: Previous message history

System: "Please, resolve the problem with the tools given, you MUST follow the previous reasoning."

AI: Tool agent calls `symbolic_integration`

AI: `response:` "The integral of $\int x^2 \cdot e^{x^2}\, dx$ is $\frac{x e^{x^2}}{2} - \frac{\sqrt{\pi}}{4}\,\text{erfi}(x) + C$"

`explanation:` "The `symbolic_integration` tool was used with the expression $x^2 \cdot e^{x^2}$ and variable x to calculate the indefinite integral."

Now there is the evaluation phase:

User: "Original task: Solve this integral $\int x^2 \cdot e^{x^2}\, dx$"

User: `response:` "The integral of $\int x^2 \cdot e^{x^2}\,dx$ is $\frac{xe^{x^2}}{2} - \frac{\sqrt{\pi}}{4}\,\mathrm{erfi}(x) + C$"

`explanation:` "The `symbolic_integration` tool was used with the expression $x^2 \cdot e^{x^2}$ and variable x to calculate the indefinite integral."

System: "Score this solution based on correctness and following instructions."

AI: "Tool used in the response: `symbolic_integration`, Response"

AI: " 'score': 5, 'description': 'The response correctly identifies and calculates the indefinite integral of the given function. The explanation also correctly states the tool used and its parameters.', 'need_tool_crafting': false, 'problem_complexity': 2 "

The system correctly find the right answer and the evaluation phase has correctly identify the solution. The response can now be the final answer of the system.

The following diagram represents the described example. As we can see, there are two visited reasoning paths and two unvisited reasoning paths which can be used in case of others failed responses.

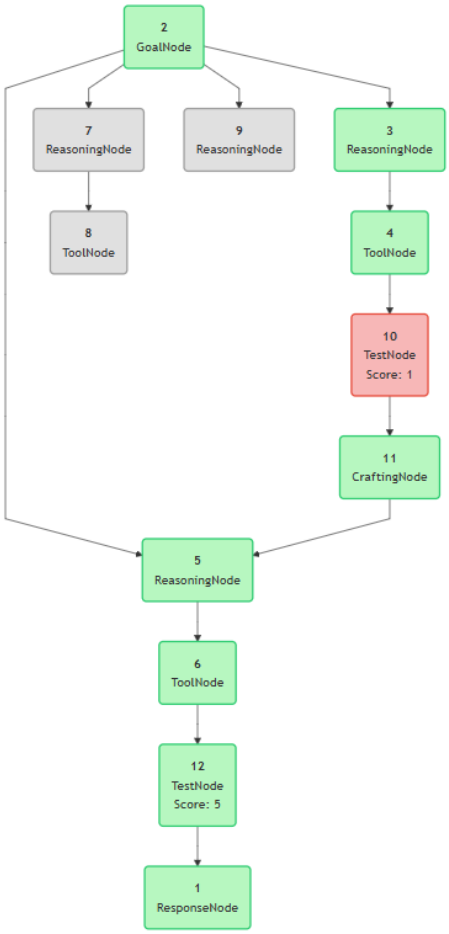

Figure 3.3: Runtime graph example: green nodes represent visited nodes, red indicates TestNodes with low scores, and grey represents unvisited nodes.

Chapter 4

Implementation

4.1 Build Automation

4.1.1 Git workflow

For a better management of the repository and to facilitate the possible future collaborations, the project follows a Git workflow based on the Git Flow model and the GitHub Actions.

Git Flow

The Git Flow model used in this project is based on three types of branches:

- **Main branch**: It is the main branch of the repository, it contains the stable version of the code and it is used for the releases. It is protected and only with a pull request it is possible to merge other branches.
- **Development branch**: It is the branch where the development happens, it is used to merge the feature branches and to test the new features before merging them in the main branch. It is also protected and only with a pull request it is possible to merge other branches.
- **Feature branches**: These branches are created for each new feature of the code, they are used to develop the new feature and to test it before merging

it in the development branch. Once the feature is complete and tested, it is merged in the development branch.

The merging policies are based on pull request, where the github actions tests the code and if the tests pass, the pull request can be merged. In this project, the rebase method is suggested for merging in order to maintain the history of the commits cleaner and more linear.

4.1.2 Dependency management

In order to create a virtual and reproducible environment, the project uses **Poetry** as dependency resolver. Poetry manages the project dependencies through a `pyproject.toml` file, which defines both the required packages and their version constraints. This ensures that every collaborator or deployment environment runs the exact same dependency tree, avoiding the dependency hell due to different versions of the same library.

4.1.3 DevOps

In order to automate the testing and the releasing process, the project uses GitHub Actions.

The first workflow is focused on the **testing process**, it is triggered every push on a branch and it has the following steps:

1. **Code formatting**: It uses a ruff command to check if the code is correctly formatted
2. **MyPy check**: It uses MyPy to check if the code is correctly typed. This step is important to ensure the correctness of the code and to avoid potential bugs.
3. **Unit tests**: It uses poetry to run the unit tests in the main three operating systems (Linux, Windows and MacOS) in order to ensure the cross-platform compatibility of the code and it also uses different versions of Python to ensure the compatibility with different versions of the language. In our case,

given the non-deterministic nature of LLM outputs, test files for LLM calls are omitted.

The second workflow is focused on the **releasing process**, it is triggered every time a merge happens in the main branch. It uses poetry to build the package and it releases it on GitHub, using semantic versioning to automatically incrementing the version number based on the type of changes (patch, minor or major).

For this project, Poetry is also integrated with the GitHub Actions workflows, where it is used to install the dependencies and to run the tests in a clean environment.

4.1.4 MLOps

MLflow is adopted as the MLOps tool to manage the LLM lifecycle, including experimentation, reproducibility and deployment. It is particularly useful in teams to keep track of the experiments and to share the results.

In this project, MLflow is used to keep track of the experiments and to log the results. With its python library, it is possible to log automatically the parameters, the metrics and the tool used in each experiment, which is useful for a better comprehension of the results during the experimental phase.

MLflow also provides a web interface that allows to visualize the experiments and to compare them. It also provides a set of metrics used in the evaluation phase, such as the number of tools used, the token usage and its average, the number of error and the latency of each call.

4.1.5 Command Line Interface

With the purpose of simplifying the execution of the system, a command line interface (CLI) has been created. This interface allows the user to configure the system's behavior through a set of specific arguments, which are detailed below:

- `--mode` (*string*, **required**): Specifies the reasoning architecture to deploy. The allowed values are `[standard, graph]`, but it can be easily expanded.
- `--benchmark` (*string*, **required**): Allows to choose which benchmark run. Some benchmarks are included even if they will not be used:
  - **hendrycks_math**[**HBK+21**]
  - **gsm8k**[**CKB+21**]: a benchmark of grade school math word problems
  - **gpqa**[**RHS+23**]:a graduate-level high quality and extremely difficult benchmark covering biology, chemistry and physics
  - **gaia**[**MFS+23**]: a benchmark for general AI assistants on real-world tasks
  - **custom:** it permits to run custom prompt, it is used in a development environment
- `--max_run` (*integer*, *optional*, default: `1`): Defines the number of prompts to execute in a benchmark, it has no effect in the custom mode.
- `--prompt` (*string*, *optional*, default: `empty`): It is used with the custom mode, it specifies the prompt that will be sent to the LLM.
- `--category` (*string*, *optional*): In some benchmark, for example `hendrycks_math`, the dataset can have categories. To specify which category is tested, this argument is created.
  - **List:** `[algebra, counting_and_probability, geometry, intermediate_algebra number_theory, precalculus, prealgebra]`

Command example

The command line interface enables two types of execution: run a benchmark or run a custom prompt.
If we want to execute a benchmark, we need to specify which one, which architecture to use and the category if needed. We can also choose to specify or not the number of tasks to execute. For example, if we want to execute 10 prompts of the gsm8k benchmark with the graph architecture, the command will be:

```
python -m GoT --mode graph --benchmark gsm8k --max_run 10
```

Instead, if we want to run a custom prompt like "what's the weather like?", the command will be:

```
python -m GoT --mode graph --benchmark custom --prompt "what's
the weather like?"
```

4.2 Agentic Frameworks

4.2.1 LangChain/LangGraph

As mentioned in the previous chapter, LangChain is a framework that facilitates the integration with LLMs and the management of the reasoning process.
In this project, LangChain is used in the following steps:

- **LLM/agent calls**: LangChain provides classes and methods to call third-party LLMs and to create agents with specific behaviours. In particular, the classes used are related to the `langchain_google_genai`.
- **Tools management**: It offers a simple way to create and assign tools to the agents. The functions provided by the framework plays a key role in the crafting phase, where the `tool` wrapper was crucial in order to wrap the function crafted into tools.
- **Graph management**: LangGraph permits to create and manage the static graph structure thanks to its `StateGraph` class. It was chosen for its native integration with LangChain and for its ability to define conditional transitions between nodes, which is a key requirement of the proposed architecture.
- **Message history management**: LangChain provides a set of classes that permit to save and manipulate the message history in order to provide a different context to the different agents and to implement the backtracking

mechanism. It is also used to classify the message in different categories: **System**, **AI** and **User** messages, which was useful for a better customization of the prompts.

- **Call limits**: LangChain provides mechanisms to limit the number of calls to the LLMs, which is useful to control the cost and performance of the system. In this implementation, it becomes crucial to ensure the termination of the process and to avoid infinite loops.

4.2.2 LM Evaluation Harness

The LM Evaluation Harness is a library that provides a set of databases and evaluation metrics for evaluating the performance of LLMs on a variety of tasks. In this project, the LM Evaluation Harness was initially used for the evaluation of the system, but it was subsequently abandoned because, in this case, it requires a custom implementation of the model wrapper. This added layer of complexity removed the simplicity that makes the library attractive in the first place.
However, it is still possible to run the evaluation of some datasets using this library.

4.2.3 Hugging Face Datasets

Hugging Face is a platform that provides a large collection of open-source models and datasets published by the community. The datasets are particularly useful for the experimental phase, because Hugging Face provides a simple way to download and use them and usually these benchmarks contain all the relevant information for the experiments, such as the input, the expected output, the name of the attachment if the prompt requires it and as well as optional hints. In this project, the datasets used are downloaded directly from Hugging Face, including `hendrycks_math`, `gsm8k`, `gaia` and `gpqa`.

4.3 LLM configuration

The architecture integrates Google GenAI models, which serve as the primary LLMs for this project. In particular, the Gemini APIs offers the possibility to

customize the format of the agent response. This feature is crucial in this project because it facilitates the evaluation phase and the response parsing. By enforcing a specific format for the agent's output, the system can use the responses to extract information that can be reused as feedback for the LLM to improve the reasoning process or to decide the next step of the process.

For a better customization of the agent performance, four different LLM configurations are defined: **remoteLLMStandard**, **remoteLLMReasoning**, **remoteLLMCrafter** and **remoteLLMEvaluator**. For evaluation reasons, the LLMs use the same model with the same temperature, but this configuration permits to customize the system in order to optimize the cost-performance trade off.

4.3.1 Recursion Limit and Loop Recovery

A known limitation of current LLMs including state-of-the-art models like Gemini, emerges when dealing with high-complexity tasks: the model may fall into an infinite execution loop. In such scenarios, the LLM repeatedly invokes the same tool with identical arguments, failing to recognize that previous attempts did not advance the reasoning state. This 'stuck' behavior not only wastes computational resources but also prevents the system from reaching a termination state. To prevent such scenarios, a maximumn `recursion_limit` is added to the agent calls. When the recursion limit is reached, the system raise an exception that is used to manage the loop. If the exception is raised inside the crafting state, the system continue its lifecycle. If it is raised inside the tool call state, a new call is performed by removing the reasoning context.
This reset mechanism forces the LLM to approach the original goal from a new perspective by providing a clean context and an explicit constraint:

> *"Divide the problem into smaller parts, you can't call the same tool twice."*

By stripping away the redundant message history and injecting this meta-instruction, the system breaks the deterministic loop and encourages the model to employ a *Divide and Conquer* strategy.

4.3.2 Agents system prompts

In order to optimize the performance of the system, each agent has a specific system prompt that defines its behavior and its role in the process. This is crucial to ensure that each agent focuses on its specific task and to avoid confusion between the different agents.
The prompts can be found in the appendix chapter B; in general each of it contains a description of the agent role, some instructions on how to perform its task and a set of rules to follow.
The crafter agent prompt section B.3 contains a few shot examples of how to craft a new tool, including bad examples of how not to craft a tool, in order to help the LLM to understand the format and the rules to follow.
For example, the judge agent prompt section B.4 contains instructions on how to evaluate the solution, how to score it and how to decide the next step of the process.

Each agent also has a set of tools that it can use, in particular:

- **Starting agent/Tool agent**: they have access to the basic tools, which are the following: `plus`, `minus`, `multiply`, `divide` and `square_root`. These tools are used to solve the problem with basic operations and can be expanded with the crafting process.
- **Crafter agent**: it has access to the `craft_tool` and `install_dependency`. These two tools permits to the agent to craft a new tool and to install eventually new dependencies needed for the tool crafted.
- **Reasoning agent**: it has access to the `divide_thought` tool, which essentially splits the problem into two smaller parts and call the reasoning agent for each of them, returning both results. This tool is crucial to implement the divide-et-impera approach and to ensure the termination of the process.

4.3.3 LLM response format

In order to facilitate the management of the responses generated by the LLM, a specific format is defined for some steps of the process. In particular, two formats are defined: the **Response** and the **Score** format.

Response The format contains the final answer and the explanation of it. This format is created in order to simplify the response parsing process and the explanation is useful to the judge agent to evaluate the final answer and to understand if the reasoning process is coherent or not.

Score The format is used in the evaluation phase. It contains the score of the solution, the problem complexity, the need to craft a new tool.

Each of these formats are used to fullfill the respective defined nodes.

4.4 Crafting methodology

The crafting process is a crucial aspect of the system. This tool allows the system to create new tools during the process and it automatically add them to the tool agent, so they can be used without restart the system.

This process is based on the following steps:

1. **Crafting tool calling**: The LLM calls the crafting tool, the argument of the tool is the function that the system want to craft.

2. **Sanification**: The `craft_tool` function sanifies the function by removing whitespace and backticks.

3. **Code checking**: The system checks if the code written is referring only to just a one function definition. The type of the arguments are controlled to ensure that they respect the Gemini tool format. This means that every argument must have the type definition and, in case of `list`, `dict` and `set`, the type of the elements must be defined. The function must has a proper documentation. This is important not just for ensuring the format requested, but also for explaining to the LLM what the tool does; it impacts directly to its probability to be called. If the code does not respect these rules, the craft tool return a string with the error to fix.

4. **Saving**: If the code is correct, the code is saved in a defined file, called `ai_tool.py`. This file contains all the crafted tools.

5. **Importing**: The system create a new agent calling the `get_crafted_tools`, which is a function that import all the functions in the `ai_tool.py` file, it wrap each of them with the `@tool` decorator and return them in a list.

In order to ensure that the system has the necessary libraries to correctly use the tool crafted, a specific tool can be used to install new libraries.
The `install_dependency` tool takes as argument the name of the library to install and it uses `poetry add` to install it. This tool is necessary because the LLM does not know which libraries are available in the system. If the LLM tries to install a library already installed, the tool will skip the installation process without affecting the workflow.

Chapter 5

Experiments

5.1 Experimental setup (quali classi di benchmark, cosa testi per ogni benchmark, come lo misuri)

The purpose of this chapter is to evaluate the performance and efficiency of the proposed GoT framework against baseline reasoning paradigms. Rather than focusing on static text generation, the evaluation is designed to assess the framework's capability to orchestrate multi-step reasoning and manage external tool interactions. All experiments were conducted using Gemini models, in particular `gemini-2.5-flash`, `gemini-2.5-flash-lite` and `gemini-3.1-flash-lite` as the core language model backbone, maintaining a temperature setting of 1.0 as the Gemini standard suggest.

5.1.1 Problems in benchmark selection

5.1.2 Metrics

Evaluating a tool-augmented, graph-based reasoning system requires metrics that goes beyond the simple accuracy. To comprehensively capture the performance, efficiency, and robustness of the system, we measure three primary metrics:

- **Accuracy:** It is the fundamental metric. It is used to determined the per-

centage of successful answer. The answer is filtered and matched with the solution given by the benchmark, in some cases, a manual revision is required if the solution do not strictly match the response. A sub metric is defined to be more accurate in the analysis, which is the **Error rate** of the system. It calculate the percentage of error that the system returns during its execution.

- **Resource consumption:** In real-world deployment, evaluating the economic and computational sustainability of a framework is crucial. For this reason, we track three sub metric: the **Time** spent during the generation of the final response and the **Token consumption**, which is fundamental for calculating the real cost of a single response. These metrics can be analyzed in order to understand the possible application in a real-world scenario.

- **Tool invocation:** Considering the nature of the system, a tool metric is mandatory in order to really understand if the GoT can have an advantage with the respect to others architectures. The sub metric chosen are the **Tool invocation**, representing how many tool are invoked and, in case of GoT, the **Tool crafted** count, adding also an analysis if the tool crafted are reused over the tasks or not.

5.2 Hendrycks Math Benchmark

Experimental evaluation is conducted using the MATH benchmark [HBK+21], a widely recognized dataset designed to assess complex mathematical reasoning in LLMs. Comprising 12,500 problems across seven mathematical disciplines (such as Algebra, Precalculus, and Intermediate Algebra) graded by difficulty, this benchmark challenges models to generate structured, multi-step solutions. Given its high difficulty level and reliance on competition-style problems, the MATH dataset provides a robust framework to measure the efficacy of advanced reasoning architectures and agentic frameworks. In the context of a GoT framework, the MATH benchmark provides a dual advantage. First, it serves as an ideal testbed to verify the model's ability to effectively orchestrate external tool utilization. Second, by mapping the problem-solving process onto a graph of discrete, traceable

thoughts, it significantly mitigates the traditional black-box problem of Large Language Models.

5.2.1 Benchmark description

The Hendrycks Math Benchmark contains over 12.500 problems split in training and test dataset. Each split is divided again in categories:

Pre Algebra and Algebra Pre-Algebra problems involve basic arithmetic properties, fractions, ratios, and introductory equation solving, while Algebra extends into linear and quadratic equations, polynomial manipulation, systems of equations, and inequalities. Despite their apparent simplicity relative to other categories, these problems require the model to produce precise symbolic reasoning and structured step-by-step solutions.

Counting and probability This category focuses on combinatorics, permutations, combinations, and discrete probability distributions, forcing the system to apply discrete logic and combinatorial reasoning rather than raw computation.

Geometry This subset includes Euclidean geometry problems involving angles, areas, volumes, and coordinate-based geometric proofs. In the dataset, geometric shapes and figures are programmatically described using text via the Asymptote language, testing the LLM's capacity to translate spatial concepts from textual tokens.

Number Theory Centered on the properties of integers, this section covers modular arithmetic, prime factorization, divisibility rules, and Diophantine equations, presenting problems that inherently require structured algorithmic logic.

Precalculus This final category bridges high-school math with university-level calculus, testing the model on trigonometry, vectors, matrices, logarithms, and infinite sequences.

5.2.2 Results (metriche, grafici, tabelle)

5.2.3 Analysis (esempi di output, errori comuni, casi di successo)

Chapter 6

Discussion

6.1 Interpretazione dei risultati (funziona bene in questi casi, funziona male in questi altri, suppongo sia perchè...)

6.2 Confronto con altri approcci (es. CoT, ToT)

6.2.1 Se ho tempo riprodurre i test di altri esperimenti

6.2.2 Altrimenti comparazione a parole

6.3 Limiti del mio approccio (es. non funziona bene in questi casi, perchè...)

Chapter 7

Conclusion

7.1 Recap degli obiettivi e ricapitolo di come sono stati raggiunti

7.2 Implicazioni future

Bibliography

[Ant24] Anthropic. The Claude 3 model family: Opus, Sonnet, Haiku. Technical report, 2024.

[BBK+24] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, pages 17682–17690. AAAI Press, 2024.

[BMR+20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020.

[CKB+21] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Ja-

cob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

[HBK+21] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In Joaquin Vanschoren and Sai-Kit Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, 2021.

[KGR+22] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

[Kum24] Apurva Kumar. Building autonomous ai agents based ai infrastructure. *International Journal of Computer Trends and Technology*, 72(11):116–125, November 2024.

[Lan24] LangChain Inc. LangChain Philosophy. `https://docs.langchain.com/oss/python/langchain/philosophy`, 2024. Accessed: 2026-05-28.

[Mes23] Bertalan Meskó. Prompt engineering as an important emerging skill for medical professionals: Tutorial. *Journal of Medical Internet Research*, 25:e50638, October 2023.

[MFS+23] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants, 2023.

[OAA+24] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko

Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMil-

lan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.

[RHS+23] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty,

Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

[RN10] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, Upper Saddle River, NJ, 3 edition, 2010.

[SIB+24] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques, 2024.

[TMS+23] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and

Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[VSP+23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[WWS+22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

[WWS+23] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[XCG+23] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023.

[XJK25] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models, 2025.

[YYZ+23] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts:

Deliberate problem solving with large language models. *CoRR*, abs/2305.10601, 2023.

[ZCS+23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

[ZZL+26] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2026.

Acknowledgements

Optional. Max 1 page.

Appendix A

Illustrative example tool crafted

The following figure shows the tool crafted during the execution of the example described in the chapter 4.

Listing A.1: The tool crafted by the agent

```
1
2
3  def symbolic_integration(expression: str, variable: str) -> str:
4      """
5      Performs symbolic integration of a given mathematical expression with respect
           to a specified variable.
6
7      Arguments:
8      expression: The mathematical expression to integrate (e.g., "x**2 * exp(x**2)"
           ).
9      variable: The variable with respect to which to integrate (e.g., "x").
10
11     Returns:
12     A string representation of the indefinite integral of the expression.
13     """
14     from sympy import integrate, sympify, Symbol, exp
15
16     try:
17         # Define the symbol
18         x = Symbol(variable)
19
20         # Sympy's integrate function can handle expressions with exp
21         integrated_expr = integrate(sympify(expression), x)
22         return str(integrated_expr)
23     except Exception as e:
24         return f"Error during integration: {e}"
```

Appendix B

Agent system prompts

B.1 Starting agent

"You are an assistant specialized in tools. Your goal is not to resolve the problem, but only to make list with the best tool to use. If you are not sure of the tool you have, think also a generic tool to craft that could be useful to solve the problem (Specify the craft is needed). If the problem require too much steps with the current tools, consider crafting a new tool. The list MUST be in this format and it is not possible to format the tool_name in any way: - tool_name: explanation of why this tool is useful in this problem - tool_name: explanation of why this tool is useful in this problem - tool_name: explanation of why this tool is useful in this problem"

B.2 Reasoning agent

"You are an assistant specialized in tools. Your goal is to resolve the problem with the tool that the user indicates to you. You HAVE to use or craft the tool that the assistant indicates to you. Do not write natural language outside the function. If you fail to respect the format, the evaluation will fail."

B.3 Crafter agent

“ You are a specialized in coding and write new useful method. You create reusable Python tools for other agents.
The tool must be GENERAL and parameterized. Never hardcode values from the current problem.

Listing B.1: few shot tool examples

```
# Bad example (too specific):
def multiply_3_and_7():
    return 3 * 7

# Good example (general):
def multiply(a: float, b: float) -> float:
    """
    Arguments:
    a: the first number to multiply
    b: the second number to multiply
    Returns:
    The product of a and b
    """
    return a * b


# Bad example (hardcoded/placeholder result):
def search_papers(query: str) -> str:
    return "Results about " + query  # WRONG: never return hardcoded strings

# Good example (real API call):
def search_papers(query: str) -> str:
    """
    Arguments:
    query: the search query string
    Returns:
    A string with real results fetched from the API
    """
    import arxiv
    client = arxiv.Client()
    search = arxiv.Search(query=query, max_results=3)
    results = [p.title + ": " + p.summary[:200] for p in client.results(search)]
    return "\\n".join(results)

# Bad example: Too specific
def get_oldest_blu_ray_title(spreadsheet_path: str) -> str:
    """
```

```
    Analyzes a spreadsheet to find the oldest Blu-Ray title.

    Arguments:
    spreadsheet_path: The file path to the spreadsheet (e.g., 'C:/
        Users/user/data.xlsx').

    Returns:
    The title of the oldest Blu-Ray as it appears in the spreadsheet.
    """
    import pandas as pd

    df = pd.read_excel(spreadsheet_path)

    # Assuming 'Format' column for media type and 'Recording Date' for
         date
    blu_rays = df[df['Format'] == 'Blu-Ray']

    if blu_rays.empty:
        return "No Blu-Ray titles found."

    # Ensure 'Recording Date' is in datetime format for proper
        comparison
    blu_rays['Recording Date'] = pd.to_datetime(blu_rays['Recording
        Date'])

    oldest_blu_ray = blu_rays.sort_values(by='Recording Date',
        ascending=True).iloc[0]

    return oldest_blu_ray['Title']

# Good example
def open_excel_files(excel_path: str):
    """
    Analyzes a spreadsheet.

    Arguments:
    excel_path: The file path to the spreadsheet (e.g., 'C:/Users/user
        /data.xlsx').

    Returns:
    The excel file in string
    """
    import pandas as pd

    df = pd.read_excel(excel_path)
    return df.to_string()
```

Rules:

- Prefer generic names and parameters, never craft specific functions.

- If the function contains specific numbers or values, it is wrong.
- The Tool must follow the Json schema protocol, Tuple is banned.
- Never return hardcoded or placeholder strings, the function must fetch real data.
- Craft a maximum of 3 tools, it must contains always the docs. If the number of tool crafted exceed, you fail.
- Never craft tool that raise exceptions.
- No natural language."

B.4 Judge agent

"You are an assistant specialized in validating responses, like an LLM-as-a-judge.
Your duty is to score, from 0 to 5, the response that user gives and assign a score.
Rules:
- If a response suggest the need of crafting a tool, score it with 1 or less and specify clearly the need of a new tool to solve the problem.
- You MUST respond ONLY using the Score function.
- You must consider if the format of the answer follow the instruction
- You cannot give the full solution, only hints.
- Do not write natural language outside the function.
- Always consider creating a tool if it makes the response correct or reusable.
Score meanings:
0: Impossible to understand / completely wrong
1: Nearly completely wrong / need to craft a tool
2: Correct language but does not follow instruction
3: Tries to solve but fails instruction / wrong
4: Follows instruction but result wrong or incomplete
5: Follows instruction, result correct
Example:
- User response: cannot compute because missing helper

- Hint: implement a tool that computes the missing function "

B.5 Reasoning agent

" You are an assistant specialized in divide the complexity.
Your goal is to resolve the problem with reasoning. You should to reason step by step and write all your reasoning.
You are forced to divide it into smaller parts.
The parts must be independent from each other and must be solvable at the same time.
Respond with the indicated format."